ECMAScript 6 has array comprehensions similar to those you're familiar with in CoffeeScript:

```
double = function(n) {
  return n * 2;
};
numbers = [2,3,5,4,2];
var doubled = [double number for each (number in numbers)];
```

ECMAScript 6

The only syntactic difference from CoffeeScript is some brackets and parentheses:

```
double (n) ->
  n * 2
numbers = [2,3,5,4,2]
doubled = double number for number in numbers
```

Array comprehensions are nice, but they're certainly no earth-shattering new feature. You might have noticed that in CoffeeScript too; the comprehensions are nice, but they're fairly limited. That changes once iterators and generators are introduced; comprehensions become very powerful.

Consider some keyboard-handling code for a computer game. When the user presses the up-arrow, down-arrow, left-arrow, left-arrow, and down-arrow keys, then the data your program ultimately receives is an array (just like in chapter 9):

```
[UP, DOWN, LEFT, LEFT, DOWN]
```

Unfortunately, there's a catch with this array—you don't have it yet. When the game starts, you haven't received any events, so you can't use a comprehension on it because a comprehension works on values—not on values you'll get later.

What if you *could* use a comprehension for these keyboard commands? What if comprehensions worked with arrays that you don't have yet? With iterators and generators they do!

Iterators